AMR Extension of LaRT: Octree Structure and Raytrace Algorithm

Contents

1. Overview

LaRT (Lyman-alpha Radiative Transfer) is a Monte Carlo radiative transfer code for Lyman-α photon propagation in astrophysical media. The base code (`LaRT_combined`) uses a uniform Cartesian grid. `LaRT_v2.00` extends it with an Adaptive Mesh Refinement (AMR) octree grid, enabling direct use of outputs from cosmological simulation codes such as RAMSES.

AMR mode is activated with `par%use_amr_grid = .true.` in the input namelist. All photon-physics modules (scattering, raytrace, peel-off, output) are selected through Fortran procedure pointers set in `setup.f90`; the outer simulation loop (`run_simulation_mod`) and photon injection (`gen_photon_car`) are therefore unchanged.

The new source files are:

File	Role
`octree_mod.f90`	Octree data type, tree operations, cell-crossing routines
`grid_mod_amr.f90`	Grid creation, opacity normalization, frequency grid
`raytrace_amr.f90`	Photon propagation through AMR cells
`scattering_amr.f90`	Resonance and dust scattering (per-cell parameters)
`peelingoff_amr.f90`	Peel-off observer calculation
`read_ramses_amr.f90`	Reader for RAMSES binary and generic text AMR formats

2. AMR Grid Data Structure

2.1 Octree Topology

The AMR grid is represented as a *linear octree*: a flat Fortran array of cells indexed $1, 2, \ldots, N_{\rm cells}$. Cell 1 is the root, which covers the entire computational box. Each internal cell has exactly eight children obtained by bisecting the parent along all three coordinate axes.

The child octant index is encoded as

$$i_{\rm oct} = 1 + i_x + 2\,i_y + 4\,i_z, \tag{1}$$

where $i_x = 1$ if $x \geq c_x$ (the cell center), otherwise $i_x = 0$, and similarly for i_y and i_z. Hence octant indices run from 1 to 8.

2.2 Arrays Stored per Cell

The Fortran derived type `amr_grid_type` (defined in `octree_mod.f90`) stores the following arrays of size $N_{\rm cells}$ for the tree topology:

Array	Dimension	Content
`parent(:)`	$N_{\rm cells}$	parent cell index (0 for root)
`children(:,:)`	$8 \times N_{\rm cells}$	child indices; 0 = leaf
`level(:)`	$N_{\rm cells}$	refinement level (0 = root)
`cx, cy, cz`	$N_{\rm cells}$	cell center coordinates (code units)
`ch(:)`	$N_{\rm cells}$	cell half-width (code units)
`ileaf(:)`	$N_{\rm cells}$	> 0: leaf index; 0: internal
`icell_of_leaf(:)`	$N_{\rm leaf}$	cell index for each leaf
`neighbor(:,:)`	$6 \times N_{\rm cells}$	face-neighbor table (see Section 3.2)

Leaf cells (`ileaf`$(i) > 0$) carry physical data stored in separate arrays of size $N_{\rm leaf}$:

Array	Symbol	Physical meaning
`rhokap`	$\bar{\kappa}_0$	HI line-center opacity per code-unit length
`rhokapD`	$\bar{\kappa}_d$	dust opacity per code-unit length
`Dfreq`	$\Delta\nu_D$	local Doppler frequency [Hz]
`voigt_a`	a	Voigt damping parameter $a = A_{21}/(4\pi\Delta\nu_D)$
`vfx, vfy, vfz`	$\tilde{v}_x, \tilde{v}_y, \tilde{v}_z$	bulk velocity / local $v_{\rm th}$

The spectral output arrays (`Jout`, `Jin`, `Jabs`) have size N_ν (the number of frequency bins) and are also members of `amr_grid_type`. All photon-weight accumulation in AMR mode (`Jout` from raytrace escape, `Jin` from photon injection, `Jabs` from dust absorption, `Jmu` from raytrace escape) goes into the corresponding `amr_grid%*` arrays. At reduction time `output_reduce_amr` (or `output_reduce_inside_amr` for the HEALPix path) calls `MPI_ALLREDUCE` on each `amr_grid%J*` and then copies the result into the corresponding `grid%J*` alias so that the shared write-output routines see populated arrays.

2.3 Coordinate Convention

All cell positions and path lengths are stored in *code units* (kpc, pc, or whatever unit the input data use). The conversion factor `par%distance2cm` is used once when computing opacities:

$$\bar{\kappa}_0\ [\text{code-unit}^{-1}] = n_{\rm HI}\,[\text{cm}^{-3}] \times \frac{\sigma_0}{\Delta\nu_D} \times d_{\rm cm}, \tag{2}$$

where $d_{\rm cm}$ is the number of centimeters per code unit. After that, all quantities are dimensionless with respect to code units, exactly mirroring the Cartesian convention.

The box is always **centered at the origin**: all leaf coordinates lie in $[-L_{\rm box}/2, +L_{\rm box}/2]$ on each axis, and the octree root spans the same range. This is required because the peel-off geometry and the default point-source position (`xs_point = ys_point = zs_point = 0`) both assume $(0,0,0)$ to be the box center. Input readers are responsible for mapping raw data to this centered convention before passing coordinates to `amr_build_tree`.

3. Building the Octree

3.1 Tree Construction: `amr_build_tree`

`amr_build_tree` takes flat arrays of leaf-cell center coordinates (`xleaf`, `yleaf`, `zleaf`), AMR levels (`leaf_level`), and box bounds, and constructs the octree by inserting each leaf cell top-down from the root. The algorithm is as follows:

1. Initialize the root cell (index 1) with half-width $h = L_{\rm box}/2$.
2. For each leaf $\ell = 1, \ldots, N_{\rm leaf}$:
   (a) Start at the root (`icell` = 1).
   (b) For each refinement level from 0 to $\ell_{\rm leaf} - 1$:
      i. Compute the octant index $i_{\rm oct}$ using the leaf center.
      ii. If `children(`$i_{\rm oct}$`, icell)` is zero, create a new internal node with half-width $h/2$ and center offset $\pm h/2$ from the parent center.
      iii. Descend: `icell` $\leftarrow$ `children`$(i_{\rm oct}$, `icell`$)$.
   (c) Mark the reached cell as leaf ℓ: `ileaf(icell)` $= \ell$.

The array is grown dynamically (doubled) if the pre-allocated size is exceeded.

3.2 Face-Neighbor Precomputation: `amr_build_neighbors`

After the tree is built, `amr_build_neighbors` pre-computes a $6 \times N_{\rm cells}$ neighbor table. Entry `neighbor`(f, i) stores the cell index of the neighbor of cell i across face f, using the face convention:

$$f = 1{:}+x, \quad 2{:}-x, \quad 3{:}+y, \quad 4{:}-y, \quad 5{:}+z, \quad 6{:}-z. \tag{3}$$

A value of 0 means the face is on the domain boundary.
For each cell i, the routine probes the point just outside each face (offset $h \times 10^{-8}$ from the face) and descends the tree to the cell at level $\leq \ell_i$ that contains that probe point:

$$\texttt{neighbor}(f,i) = \texttt{amr\_find\_cell\_at\_level}\left(c_i + h_f \hat{e}_f,\ \ell_i\right), \tag{4}$$

where h_f is the signed probe offset along face direction f and $\hat{e}_f$ is the corresponding unit vector.
The result may be a leaf cell at the same level, a leaf at a coarser level (if the grid is not refined there), or an internal cell at the same level (if the neighbor side is *more* refined than cell i). The last case is handled at runtime by `amr_next_leaf` (Section 4.3).
This table is built once and then used for O(1) boundary crossings throughout the simulation.

4. Cell-Crossing Algorithms

4.1 Finding a Leaf: `amr_find_leaf`

`amr_find_leaf`(x,y,z) descends the octree from the root, choosing the correct child octant at each level using the octant formula (Equation 1), until it reaches a cell with `ileaf` > 0. The result is the leaf index $\ell \in \{1,\ldots,N_{\text{leaf}}\}$. The depth is at most $\ell_{\max}$, the maximum AMR level.
This function is used only at photon injection (once per photon) and as a fallback if `photon%icell_amr` is not yet set.

4.2 Cell-Exit Distance: `amr_cell_exit`

Given a photon at position (x,y,z) inside cell i with direction (k_x,k_y,k_z), `amr_cell_exit` computes the distance t_{exit} to the nearest cell face and identifies which face f is crossed.
For each axis $\alpha \in \{x,y,z\}$ with cell center c_α and half-width h, the distances to the $\pm$ faces are:

$$t_\alpha^+ = \frac{c_\alpha + h - \alpha}{k_\alpha}, \quad t_\alpha^- = \frac{c_\alpha - h - \alpha}{k_\alpha}, \tag{5}$$

with $t = \infty$ (implemented as `hugest`) when the direction component is zero or points away from the face. The exit is the minimum over all six half-space times:

$$t_{\text{exit}} = \min(t_x^+, t_x^-, t_y^+, t_y^-, t_z^+, t_z^-), \tag{6}$$

and f is the index of the minimizing element (1–6 in the convention above).

4.3 Next Leaf After a Face Crossing: `amr_next_leaf`

After the photon crosses face f of cell i and the position has been updated to (x,y,z) at the face, `amr_next_leaf`(i,f,x,y,z) returns the leaf index of the cell the photon enters. The algorithm is:

1. Look up `ineigh` = `neighbor`(f,i). If zero, the photon has left the domain; return 0.
2. If `ileaf(ineigh)` > 0, the neighbor is already a leaf; return `ileaf(ineigh)`.
3. Otherwise the neighbor is an internal cell at a finer level. Descend:

   (a) Compute $i_{\rm oct}$ from the actual crossing position (x, y, z) and the center of `ineigh`.

   (b) `ineigh` $\leftarrow$ `children`$(i_{\rm oct}$, `ineigh`$)$.

   (c) Repeat until `ileaf(ineigh)` > 0.

4. Return `ileaf(ineigh)`.

Step 1 is O(1); the descent in step 3 traverses at most $\Delta\ell = \ell_{\rm max} - \ell_i$ extra levels, which is typically one or two. In practice this makes cell-boundary crossings effectively O(1) for any AMR grid.

5. Grid Initialization

`grid_create_amr` in `grid_mod_amr.f90` performs the following sequence:

1. **Read leaf-cell data.** MPI rank 0 reads the data file (RAMSES binary or generic text) and broadcasts all arrays to the other ranks. Each rank therefore holds a full copy of the tree (appropriate for moderate grid sizes).

2. **Build octree.** Call `amr_build_tree` and `amr_build_neighbors`.

3. **Compute per-cell Doppler and Voigt parameters.** For leaf ℓ:

$$\Delta\nu_{D,\ell} = \frac{v_{\rm th,1}\sqrt{T_\ell}}{\lambda_0}, \quad a_\ell = \frac{A_{21}}{4\pi\,\Delta\nu_{D,\ell}}, \tag{7}$$

   where $v_{\rm th,1} = \sqrt{2k_B/m_H}$ is the thermal velocity per $\sqrt{T}$.

4. **Compute per-cell opacity and velocity.**

$$\bar{\kappa}_{0,\ell} = n_{{\rm HI},\ell}\,\frac{\sigma_0}{\Delta\nu_{D,\ell}}\,d_{\rm cm}, \tag{8}$$

$$\tilde{v}_{\alpha,\ell} = \frac{v_{\alpha,\ell}[{\rm km\,s^{-1}}]}{v_{{\rm th},\ell}[{\rm km\,s^{-1}}]}, \tag{9}$$

   where σ_0 is the hydrogen Lyman-α cross-section at line center and $d_{\rm cm}$ is centimeters per code unit.

5. **Normalize opacity to `taumax`.** See Section 5.2.

6. **Set up frequency grid and output arrays.**

5.1 Input Data Formats

Two readers are provided.

RAMSES format (`amr_type = 'ramses'`). Reads the standard RAMSES AMR and hydro binary output files from `output_XXXXX/`. Unit scales (`unit_l`, `unit_d`, `unit_t`) are taken from `info_XXXXX.txt`. The routine returns leaf positions in physical units (cm). RAMSES stores fractional positions in $[0, 1]$ (fraction of `boxlen`); the reader maps these to the centered convention as

$$x_{\rm leaf} = \left(x_{\rm RAMSES} - \tfrac{1}{2}\right) L_{\rm box}, \tag{10}$$

so that the output positions lie in $[-L_{\rm box}/2, +L_{\rm box}/2]$.

Generic text format (`amr_type = 'generic'`). A plain-text file with a header followed by one line per leaf cell:

```
# comment lines start with #
BOXLEN  <box_length_in_code_units>
NLEAF   <number_of_leaf_cells>
# x  y  z  level  nH[cm^-3]  T[K]  vx[km/s]  vy[km/s]  vz[km/s]
<x>  <y>  <z>  <level>  <nH>  <T>  <vx>  <vy>  <vz>
...
```

The code unit is set by `par%distance_unit` (e.g., `'kpc'`). Leaf coordinates must be centered at the origin, i.e., in $[-L_{\rm box}/2, +L_{\rm box}/2]$ on each axis. The Python `AMRGrid` class writes coordinates in this convention. When reading a FITS binary table, the `ORIGINX`/`ORIGINY`/`ORIGINZ` header keywords (written by `AMRGrid` as $-L_{\rm box}/2$) are *informational only* and are **not** used to shift the coordinates.

5.2 Tau Normalization by Pole Traversal

The optical depth normalization convention in `LaRT_v2.00` matches the Cartesian version: `taumax` is the line-center optical depth from the box center to the $+z$ boundary along the z-axis.

After the raw opacities are set, a ray is traced from the box center in the $+z$ direction using the same AMR raytrace kernel:

$$\tau_{\rm pole} = \sum_{\ell} \bar{\kappa}_{0,\ell}\, \phi(0, a_\ell)\, \Delta s_\ell, \tag{11}$$

where $\phi(x, a)$ is the Voigt profile at $x = 0$ and Δs_ℓ is the path-length through leaf ℓ along the ray. The normalization factor is

$$f_{\rm norm} = \frac{\texttt{taumax}}{\tau_{\rm pole}}, \tag{12}$$

and all opacities are scaled: $\bar{\kappa}_{0,\ell} \leftarrow f_{\rm norm}\, \bar{\kappa}_{0,\ell}$. The same factor is applied to dust opacity if present.

This pole-traversal approach is exact for any opacity distribution, in contrast to the volume-average approximation $\tau \approx \langle \bar{\kappa}_0 \rangle L_{\rm box}$ that underestimates $\tau_{\rm pole}$ for centrally concentrated clouds such as a sphere.

6. Photon Propagation

6.1 Main Raytrace Routine: `raytrace_to_tau_amr`

This routine propagates a photon through the AMR grid until it accumulates a target optical depth $\tau_{\rm in}$ (sampled as $-\ln\xi$ for a uniform random ξ) or leaves the box.
The photon state is represented by:

- (x, y, z) — current absolute position in code units,
- (k_x, k_y, k_z) — unit direction vector,
- x_ν — frequency offset in units of the local Doppler width (comoving frame of the current cell),
- ℓ — current leaf index (`photon%icell_amr`).

The loop at each step n:

1. Convert leaf index to cell index: i = `icell_of_leaf`(ℓ).
2. Compute the exit distance and exit face: $(t_{\rm exit}, f)$ = `amr_cell_exit`$(i, x, y, z, k_x, k_y, k_z)$.
3. Evaluate the opacity at the current frequency:

$$\kappa = \bar{\kappa}_{0,\ell}\,\phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell} \quad \text{(with dust, if present)}, \tag{13}$$

   where $\phi(x, a)$ is the Voigt function.
4. Update accumulated optical depth: $\tau \leftarrow \tau + \kappa\, t_{\rm exit}$.
5. **If** $\tau \geq \tau_{\rm in}$: backtrack to the exact interaction point (Section 6.3) and exit the loop.
6. Otherwise: advance position to the face,

$$(x, y, z) \leftarrow (x + t_{\rm exit}\,k_x,\ y + t_{\rm exit}\,k_y,\ z + t_{\rm exit}\,k_z). \tag{14}$$

7. Look up the next leaf: ℓ' = `amr_next_leaf`(i, f, x, y, z).
8. If $\ell' = 0$, the photon has escaped; set `photon%inside = .false.` and exit.
9. Apply the frequency shift (Section 6.2): $x_\nu \leftarrow x'_\nu$.
10. Set $\ell \leftarrow \ell'$ and continue.

6.2 Frequency Shift at Cell Boundaries

Each cell has its own Doppler frequency $\Delta\nu_{D,\ell}$ (because cells can have different temperatures) and its own bulk velocity component along the photon direction,

$$u_\ell = \tilde{v}_{x,\ell}\,k_x + \tilde{v}_{y,\ell}\,k_y + \tilde{v}_{z,\ell}\,k_z \quad \text{[dimensionless, in units of local } v_{\rm th}\text{]}. \tag{15}$$

When crossing from cell ℓ (old) to cell ℓ' (new), the photon frequency must be transformed to the comoving frame of the new cell. The physical frequency is invariant, so:

$$\nu_{\rm phys} = \nu_0 \left(1 + \frac{(x_\nu^{\rm old} + u_\ell)\,\Delta\nu_{D,\ell}}{\nu_0}\right), \tag{16}$$

and the new dimensionless frequency is:

$$\boxed{x_\nu^{\rm new} = (x_\nu^{\rm old} + u_\ell)\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\ell'}} - u_{\ell'}.} \tag{17}$$

This formula transforms between the comoving frames of two cells with different temperatures and bulk velocities in a single step.

6.3 Overshoot Correction

When $\tau > \tau_{\rm in}$ after stepping across a cell, the photon actually stopped inside the cell. The exact interaction distance is found by backtracking:

$$\Delta s_{\rm back} = \frac{\tau - \tau_{\rm in}}{\kappa}. \tag{18}$$

The total displacement from the entry point is then $d = d + t_{\rm exit} - \Delta s_{\rm back}$, and the final position is

$$(x, y, z) \leftarrow (\texttt{photon\%}x + d\,k_x,\ \texttt{photon\%}y + d\,k_y,\ \texttt{photon\%}z + d\,k_z). \tag{19}$$

Note that the position update is accumulated as a single floating-point addition from the *original* entry point (not from successive face crossings), which preserves numerical precision.

6.4 Photon Escape and Spectrum Accumulation

When the photon leaves the box ($\ell' = 0$), the frequency is converted to the lab frame:

$$x_\nu^{\rm lab} = x_\nu + u_\ell, \tag{20}$$

then scaled to the reference Doppler frequency:

$$x_\nu^{\rm ref} = x_\nu^{\rm lab}\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\rm ref}}. \tag{21}$$

The frequency bin index is

$$i_\nu = \left\lfloor \frac{x_\nu^{\rm ref} - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{22}$$

and the photon weight is accumulated into $\texttt{amr\_grid\%Jout}(i_\nu)$.

6.5 Boundary Handling: Periodicity and Symmetry Planes

When a photon crosses an outer face of the AMR root box, the action depends on the boundary flags set in the input file:

`par%xy_periodic = .true.` The $\pm x$ and $\pm y$ faces of the box are wrapped (periodic atmosphere mode). On exiting through face $\pm x$ ($\pm y$), the corresponding coordinate is shifted by $\mp$`amr_grid%xrange` ($\mp$`amr_grid%yrange`) and `amr_find_leaf` is called to locate the new leaf cell on the opposite face. The $\pm z$ faces remain open: a photon escaping through them is registered into `amr_grid%Jout` as in the standard outflow case.

`par%xy_symmetry = .true.` The $-x$ and $-y$ faces of the box are symmetry planes. On exiting through face 2 ($-x$) the photon's k_x is reflected ($k_x \to -k_x$) and the photon is kept inside the same leaf cell (`il_new = il`); similarly for face 4 ($-y$). The $+x$, $+y$, and $\pm z$ faces remain open.

`par%xyz_symmetry = .true.` In addition to the $-x$ and $-y$ reflections, face 6 ($-z$) is also a symmetry plane: $k_z \to -k_z$ and the photon is kept in the same cell. Only the $+x$, $+y$, and $+z$ faces remain open.

The reflection operation assumes that the AMR cell volume does *not* straddle the symmetry plane: the symmetry plane must coincide with the box edge $-$`xrange`$/2$, $-$`yrange`$/2$, or $-$`zrange`$/2$. This assumption is consistent with the way `xy_symmetry`/`xyz_symmetry` runs are constructed in the Cartesian path, where the box is set so that the symmetry plane is on a cell boundary.

Frequency transformation across the boundary. The frequency-shift formula $x_\nu^{\rm new} = (x_\nu + u_1)\Delta\nu_{D,\rm old}/\Delta\nu_{D,\rm new} - u_2$ applied at every cell crossing is replaced, on a reflection, by

$$x_\nu^{\rm new} = (x_\nu + u_1) - u_2, \tag{23}$$

where u_1 and u_2 are the projections of the local fluid velocity onto the *outgoing* (pre-reflection) and *incoming* (post-reflection) photon directions, respectively, both evaluated in the same cell. This reduces to a sign flip of the relevant velocity component.

Final position when the photon stays in the box. For `raytrace_to_tau_amr`, the position update is propagated cell by cell so that successive reflections do not corrupt the final (x, y, z). The routine keeps a running (x, y, z, k_x, k_y, k_z) and writes the final state back into `photon` only on return. For `raytrace_to_edge_*_amr` and `raytrace_to_dist_*_amr`, which take a copy of the photon and never write it back, only the local copy is updated.

6.6 Other Raytrace Routines

Six additional routines in `raytrace_amr_mod` share the same AMR traversal kernel but accumulate different quantities:

Routine	Purpose
`raytrace_to_edge_amr`	Total τ to box edge (photon unchanged)
`raytrace_to_edge_tau_gas_amr`	Gas-only τ to box edge
`raytrace_to_edge_column_amr`	$N_{\rm HI}$ and dust τ to box edge
`raytrace_to_dist_amr`	Total τ to fixed distance s
`raytrace_to_dist_tau_gas_amr`	Gas-only τ to fixed distance s
`raytrace_to_dist_column_amr`	$N_{\rm HI}$ and dust τ to fixed distance s

All six routines propagate a copy of the photon state (photon is not modified) and apply the same frequency-shift formula at each cell crossing so that the Voigt opacity is evaluated at the correct comoving frequency in every cell. They share the same boundary handling described in Section 6.5: local copies of k_x, k_y, k_z are flipped on reflection so that `photon0` itself is never modified.

7. Scattering

The scattering routines in `scattering_amr.f90` mirror the Cartesian versions but read physical parameters from `amr_grid` arrays indexed by the leaf index `photon%icell_amr`.

7.1 Resonance Scattering

`scatter_resonance_nostokes_amr` implements the resonance scattering algorithm (no Stokes polarization). The scattering is handled by the procedure pointer `do_resonance`, which is set at initialization to one of `do_resonance1_amr`, `do_resonance2_amr`, etc., depending on the line type.

1. **Sample the scattering atom velocity.** The thermal velocity component of the atom along the photon direction (in the cell comoving frame) is sampled from the redistribution function via `rand_resonance_vz`:

$$u_z = x_\nu + u_\ell. \tag{24}$$

   The two perpendicular thermal velocity components (u_x, u_y) are drawn independently from a Gaussian with unit variance.

2. **Sample the scattering angle.** The polar scattering angle θ (between the incoming and outgoing photon directions) is drawn from the phase function

$$P(\cos\theta) = \frac{3}{8}E_1\cos^2\theta + \frac{4-E_1}{8}, \tag{25}$$

   via the function `rand_resonance`(E_1). The parameter E_1 is a property of the atomic transition:

   - $E_1 = 1$: pure Rayleigh scattering ($J = 0 \to J = 1 \to J = 0$, e.g. He I).

- $0 < E_1 < 1$: mixed Rayleigh + isotropic (e.g. Lyα: $E_1 \approx 0$ for the $J = 1/2$ branch, $E_1 = 1$ for the $J = 3/2$ branch, giving an effective E_1 that depends on the occupation of upper sub-levels).
- $E_1 = 0$: isotropic scattering.
- $-1/2 \leq E_1 < 0$: oblate phase function (forward and backward directions suppressed).

The azimuthal angle ϕ is drawn uniformly from $[0, 2\pi)$.

3. **Compute the new photon frequency.** In the atom frame the frequency after scattering is

$$x'_\nu = x_\nu^{\rm atom} + u_z \cos\theta + (u_x \cos\phi + u_y \sin\phi)\sin\theta, \tag{26}$$

where $x_\nu^{\rm atom} = x_\nu - u_z$ is the frequency in the atom rest frame. An optional recoil correction $\Delta x = -g_{\rm recoil}(1 - \cos\theta)$ is applied if `par%recoil = .true.`

4. **Apply core-skip acceleration** if $|x_\nu| < x_{\rm crit}$.

The core-skip threshold $x_{\rm crit}$ is computed in `grid_create_amr` from $a\tau_0^{\rm cell}$, the product of the mean Voigt damping parameter and the mean optical depth per AMR leaf cell.

7.2 Dust Scattering

`scatter_dust_nostokes_amr` uses the standard Henyey–Greenstein phase function. When dust is present, the probability of the interaction being a dust event is

$$p_d = \frac{\bar{\kappa}_{d,\ell}}{\bar{\kappa}_{0,\ell}\,\phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell}}, \tag{27}$$

sampled at the scattering location (cell ℓ).

7.3 Multi-Level Resonance Transitions

For complex atoms (HeI, FeII) represented by multiple transitions, the AMR scattering module provides `do_resonance4_amr`, `do_resonance5_amr`, and `do_resonance6_amr`, which follow the same logic as their Cartesian counterparts but use per-cell Doppler frequencies and Voigt parameters.

8. Output Normalization

After all photons are traced, the MPI reduction collects all per-frequency contributions via `MPI_ALLREDUCE` into `amr_grid%Jout` (`output_reduce_amr` in `output_sum_rect.f90`).
The normalized specific intensity (per unit frequency per steradian) is:

$$J_{\rm out}(x_\nu) = \frac{\texttt{Jout}(i_\nu)}{N_{\rm ph}\,\delta x\,2\pi\,A}, \tag{28}$$

where $N_{\rm ph}$ is the number of photons, δx the frequency bin width, and

$$A = 4\pi\, d_{\rm cm}^2 \tag{29}$$

is the surface area of a unit sphere in cm^2 (the factor $d_{\rm cm}$ is `par%distance2cm`).
For dimensionless runs (`distance2cm = 1`, no `distance_unit`), this reduces to $A = 4\pi$, making $J_{\rm out}$ independent of the arbitrary box size. This matches the Cartesian convention ($A = 4\pi\, r_{\rm max}^2\, d_{\rm cm}^2$) when $r_{\rm max} = L_{\rm box}/2$ and the same `distance2cm` is used.

9. Core-Skip Acceleration

Lyman-α photons scatter extremely frequently in the line core ($|x_\nu| \ll 1$), where the optical depth per cell is large. The core-skip algorithm (Laursen et al. 2009) exploits the fact that a photon in the core will scatter many times within a single cell without traveling far. When $|x_\nu| < x_{\rm crit}$, the frequency is redrawn from the wing distribution directly, skipping the core-scattering steps.
In AMR mode, $x_{\rm crit}$ is computed from the mean opacity product:

$$(a\tau_0)_{\rm cell} = \bar{a}\,\bar{\tau}_0 \;/\; N_{\rm cell,axis}, \tag{30}$$

where $N_{\rm cell,axis} = L_{\rm box}/(2\,h_{\rm root})$ is the number of root cells along one axis. The threshold is then

$$x_{\rm crit} = \begin{cases} 0.02\,\exp[\xi\,(\ln(a\tau_0)_{\rm cell})^\chi] & (a\tau_0)_{\rm cell} > 1, \\ 0 & \text{otherwise}, \end{cases} \tag{31}$$

with $(\xi, \chi) = (0.6, 1.2)$ for $(a\tau_0)_{\rm cell} \le 60$ and $(1.4, 0.6)$ otherwise.

10. HEALPix Inside-Observer Mode

When the user sets `par%nside` > 0, `setup.f90` automatically enables `par%observer_located_inside = .true.` and switches the entire output and peel-off chain to HEALPix all-sky maps. This mode is intended for "galactic" observations where the observer sits at a chosen position (o_x, o_y, o_z) *inside* the medium; each escaping photon contributes to the all-sky map indexed by the direction from the observer back to the photon, computed with `vec2pix(observer%nside, ...)`.
For the AMR path, a parallel set of `_inside_amr` routines mirrors the Cartesian `_inside` chain:

Routine	File
peeling_direct_inside_amr	peelingoff_amr.f90
peeling_dust_nostokes_inside_amr	peelingoff_amr.f90
peeling_resonance_nostokes_inside_amr	peelingoff_amr.f90
output_reduce_inside_amr	output_sum_heal.f90
output_normalize_inside_amr	output_sum_heal.f90
make_sightline_tau_inside_amr	sightline_tau_heal.f90

Each peel routine computes the direction from the photon to the observer,

$$\hat{k}_{\rm obs} = \frac{\mathbf{r}_{\rm obs} - \mathbf{r}_{\rm ph}}{|\mathbf{r}_{\rm obs} - \mathbf{r}_{\rm ph}|}, \tag{32}$$

applies the Doppler shift in the local cell using $\Delta\nu_D$ = `amr_grid%Dfreq(il)` and $\tilde{v}$ = `amr_grid%vfx,y,z(il)`, traces the optical depth from the photon position to the observer with `raytrace_to_dist_amr`, and deposits the peeled weight into either `observer(i)%direc_heal[_2D]` (direct) or `observer(i)%scatt_heal[_2D]` (scattered).

Source scope. The AMR `_inside` path supports the same internal sources as the Cartesian `_inside` path: `point`, `uniform`, `gaussian`, `exponential`, `sersic`, and `diffuse_emissivity`. External-illumination geometries (`point_illumination`, `stellar_illumination`, `plane_illumination`) are only meaningful for an observer outside the medium and remain restricted to the `_outside` path. Stokes polarization is not yet implemented for the HEALPix `_inside` path (mirroring the Cartesian convention).

Normalization. `output_normalize_inside_amr` uses the AMR convention $A = 4\pi\, d_{\rm cm}^2$ (independent of $L_{\rm box}$) to normalize `Jout`, `Jin` and `Jabs`, and $A_{\rm pix} = \Omega_{\rm pix}\, d_{\rm cm}^2$ for the HEALPix observer pixels, identical to the Cartesian `output_normalize_inside` modulo the area choice.

Sightline tau. `make_sightline_tau_inside_amr` walks from the observer position to each of the six box faces ($\pm$x, $\pm$y, $\pm$z) by stepping along the relevant axis until reaching the `amr_grid%xmin/xmax/ymin/ymax/zmin/zma` boundary, accumulating τ and N(HI) via `raytrace_to_dist_tau_gas_amr` and `raytrace_to_dist_column_amr`. The outside-observer counterpart is `make_sightline_tau_outside_amr` (added 2026-05-04, in `sightline_tau_rect.f90`); the dispatch at the `par%use_amr_grid` branch in `setup_procedure` now points `make_sightline_tau` at this routine when `.not. par%observer_located_inside`. Previously the AMR + outside combination dispatched to the Cartesian `make_sightline_tau_outside`, which reads `grid%rhokap` – empty in AMR mode – so the tau output was uniformly zero. The new routine mirrors the v2.10 implementation in `sightline_tau_amr.f90`: for each pixel of the `(nxim, nyim)` detector grid it places a virtual photon at the far face of the AMR box and integrates inward toward the observer using `raytrace_to_edge_tau_gas_amr` and `raytrace_to_edge_column_amr`.

$\tau_{\rm huge}$ early-exit (sight-line vs. photon transport). The AMR raytrace routines used by the photon-transport stage early-exit when the running optical depth exceeds the double-precision $\exp(-\tau)$ underflow point ($\tau_{\rm huge} \approx 745.2$); accumulating further contributes nothing to the photon's transmission. The sight-line variants `raytrace_to_edge_tau_gas_amr` and `raytrace_to_dist_tau_gas_amr` exist precisely to *report* the integrated τ, however large, and therefore run *without* that cap (matching the Cartesian `raytrace_to_edge_tau_gas` and `raytrace_to_dist_tau_gas_car` in `raytrace_car.f90`). A short-lived bug (fixed 2026-05-04) had the cap inadvertently applied in `raytrace_to_edge_tau_gas_amr`, capping outside-observer sight-line τ at ~ 745 and producing AMR/Cartesian disagreement of an order of magnitude on the line-center maps.

Cleanup convention (`grid_destroy_amr`). `grid_destroy_amr` now mirrors the Cartesian and clump `grid_destroy` routines: it issues a single `destroy_shared_mem_all()` (idempotent when `num_windows == 0`), nullifies every shared-memory pointer field of `amr_grid`, and Fortran-deallocates only the genuinely allocatable per-rank output arrays (`Jout`/`Jin`/`Jabs`/`Jmu`). Calling `destroy_mem` on each shared-memory pointer individually (the old behavior) crashed whenever another module (`observer_destroy_outside`, `grid_destroy_clump`) had already freed the global window registry: the per-array routine fell through to a Fortran `deallocate()` of MPI-window memory and tripped ifort `severe (173)`.

Smoke test (uniform sphere, point source at origin, $T = 10^4$ K, $\tau_0 = 10^4$, observer at $o_x = 0.3$, nside = 8, 10^6 photons).

Quantity	AMR	Cartesian (64^3)
$\sum$ Jout	8.5135×10^{-2}	8.5136×10^{-2}
$\sum$ Jin	8.5135×10^{-2}	8.5136×10^{-2}
$\sum$ Scattered HEALPix	3021	3149
$\sum$ Direct HEALPix	5.13×10^{-2}	4.52×10^{-2}

The `Jout`/`Jin` agreement matches the `_outside` validation, and the $\sim 4\%$ difference in $\sum$ Scattered is consistent with the documented octree volume-averaging of the sphere boundary. The Direct HEALPix sum is dominated by very few unscattered photons at $\tau_0 = 10^4$, so its $\sim 13\%$ difference is within shot-noise expectation.

Jin accumulation in AMR. Prior to 2026-04-30, `generate_photon.f90` unconditionally wrote the injected photon weight to `grid%Jin`, but `output_reduce_amr` (and the new `output_reduce_inside_amr`) reduce `amr_grid%Jin` and then overwrite `grid%Jin` with the reduced value, losing the accumulated counts. The fix dispatches the accumulation site on `par%use_amr_grid`: in AMR mode the weight is accumulated into `amr_grid%Jin`, in Cartesian mode into `grid%Jin`, matching the existing pattern used for `Jout`, `Jabs` and `Jmu`. The smoke test above is the post-fix result.

11. Validation

The AMR implementation was validated against equivalent Cartesian runs for a uniform-density sphere of HI gas with a central point source ($T = 10^4$ K, no dust).

The AMR sphere is described by the generic text format with the same set of leaf cells at the same AMR level, providing effectively the same resolution as the 101^3 Cartesian grid. The Cartesian comparison uses $r_{\max} = L_{\rm box}/2$ with no `distance_unit`.

τ_0	$J^{\rm AMR}_{\rm out,max}$	$J^{\rm CAR}_{\rm out,max}$	Ratio (AMR/CAR)	Peak velocity
10^2	6.630×10^{-3}	6.836×10^{-3}	0.970	$\pm 26.75\,{\rm km\,s^{-1}}$
10^4	7.144×10^{-3}	7.402×10^{-3}	0.965	$\pm 38.21\,{\rm km\,s^{-1}}$

Peak positions, column densities $N({\rm HI})_{\rm pole}$, and $\tau_{\rm pole}$ agree exactly between AMR and Cartesian runs. The $\sim$ 3–5% amplitude difference is entirely attributable to the octree voxelisation of the sphere boundary: leaf cells that straddle the sphere edge carry only the average density, softening the sharp boundary. No normalization error is present.

12. Summary of Key Design Choices

1. **Flat linear octree.** All cells (internal and leaf) are stored in a single 1-indexed Fortran array. Parent–child relationships are maintained through integer index arrays. This avoids pointer indirection and is cache-friendly.

2. **Precomputed face-neighbor table.** Built once before the simulation loop. Boundary crossings cost O(1) (plus O($\Delta\ell$) descent into finer cells if the neighbor is more refined).

3. **Per-cell Voigt parameters.** Each leaf stores its own $\Delta\nu_D$ and a, supporting arbitrary temperature distributions without any approximation.

4. **Frequency shift at every cell crossing.** The comoving-frame frequency is updated at each boundary, so the Voigt opacity is always evaluated in the correct local frame, including bulk velocity effects.

5. **Single floating-point accumulation of displacement.** The photon position is updated as $\mathbf{r}_0 + d\,\hat{k}$ rather than as a sequence of increments, preserving numerical precision across many cell crossings.

6. **Procedure pointers for AMR/Cartesian switching.** The simulation loop and photon injection are unchanged. AMR physics is activated solely by reassigning Fortran procedure pointers in `setup.f90`.

7. **Pole-traversal tau normalization.** The AMR raytrace kernel itself is used to compute $\tau_{\rm pole}$, giving an exact normalization that is consistent with the Cartesian convention and avoids volume-average errors.